



WIZARD'S CHESS

A Python Chess Engine

PROJECT REPORT

Prepared by:

1. Onobun Jason Osayande: 252030164
2. Precious Ugochukwu Okwuego: 252030175
3. Obinna Moses: 242030086
4. Beatah Ubong-Colonel: 242030112

Date: March-May 2026

Instructor: Dr. Emmanuel Ali

1. Project Overview

Wizard's Chess is a fully-featured, two-player chess application implemented in Python using the Pygame library for rendering and interaction. The project delivers a polished, thematic desktop chess experience complete with an artificial intelligence opponent, animated move effects, contextual sound effects, a stylized main menu, and an in-game move log. The codebase spans four Python modules, each with a distinct responsibility, and demonstrates a clean separation of concerns between game logic, AI computation, rendering, and navigation. The game and its aesthetics take inspiration from Harry Potter: The philosopher's stone in which in the film, the trio Harry, Hermione and Ron play the game "Wizard's Chess".

Over the course of the project we were made sure to implement every thing that was taught in the lectures from github in which we saved versions of our code repeatedly to data structures in which you would see the usage of dictionaries, lists, tuples. Object oriented programming was also incredibly implemented in the project code and we also implemented a concept called "Exceptions" which we learnt from different open source platforms.

1.1 High-Level Statistics

Language	Python
GUI Framework	Pygame
Source Modules	4 (ChessEngine2.py, ChessMain.py, chessAI.py, chessMenu.py)
Approximate Lines Of Code	~1,350 lines across all modules
AI Algorithm	NegaMax with Alpha-Beta Pruning (depth-configurable)
Special Moves	En Passant, Castling (Kingside + Queenside), Pawn Promotion
Visual Theme	Dark arcane / wizard aesthetic (deep purples, gold accents)
Audio	5 contextual sound events + background menu music
Game Modes	Human vs Human, Human vs AI

2. Software Architecture

The project follows a layered architecture. The bottom layer is pure game logic; the middle layer handles AI search; and the top layer handles rendering and user interaction. A dedicated menu module bootstraps the application and passes configuration parameters to the game layer.

Module	Responsibility
--------	----------------

ChessEngine2.py	All game logic: board state, move generation, check detection, special moves, move/undo, notation
chessAI.py	AI search algorithms: MinMax, NegaMax, Alpha-Beta Pruning, evaluation functions, position tables
ChessMain.py	Pygame rendering loop, user input handling, animations, sounds, promotion UI, multiprocessing AI dispatch
chessMenu.py	Main-menu and AI-settings screens, difficulty/colour selection, music management, screen routing

2.1 Module Dependency Graph

The dependency flow is strictly top-down, which prevents circular imports and keeps the logic layer self-contained:

```

chessMenu.py
├─ imports ChessMain (launches main game loop)
└─ imports chessAI   (sets DEPTH constant before game starts)

ChessMain.py
├─ imports ChessEngine2 (GameState, Move, CastleAbility)
└─ imports chessAI      (findBestMove, randomMoveFinder, DEPTH)

chessAI.py
└─ imports (standard library only: random)

ChessEngine2.py
└─ imports (no dependencies – pure Python)

```

This design means ChessEngine2.py is fully portable and testable in isolation. The AI module only depends on the standard library, which simplifies future porting or unit testing.

3. Chess Engine (ChessEngine2.py)

This is the core of the project. It contains three classes: GameState, CastleAbility, and Move.

3.1 Board Representation

The board is represented as an 8x8 Python list. Each cell holds a two-character string: the first character is the piece colour ('w' for white, 'b' for black) and the second is the piece type. Empty squares are represented by '--'.

Piece type codes used throughout the codebase:

Code	Piece
p	Pawn
R	Rook
N	Knight
B	Bishop
Q	Queen
K	King

Example: 'wQ' is a White Queen, 'bN' is a Black Knight. The use of 'N' for Knight (rather than 'K') avoids collision with the King code, a standard convention in chess programming.

3.2 GameState Class

GameState is the central state container. It initializes the board, tracks whose turn it is, maintains a full move log, stores both kings' positions for fast check detection, and manages en passant and castling rights. Key attributes are:

- board — the 8x8 list described above
- whiteToMove — boolean flag that toggles on every move
- movelog — a list of Move objects; used for undo and the on-screen move log
- whiteKingLocation / blackKingLocation — (row, col) tuples updated on every king move for O(1) check-test lookups
- checkmate / stalemate — boolean flags set by validMovesFinder()
- enpassantPossible — stores the target square after a double pawn push
- currentCastlingRights — a CastleAbility instance, logged on every move for accurate undo

3.3 Move Generation

Move generation is split into two phases:

1. getAllPossibleMoves() — iterates every square, identifies the current player's pieces, and dispatches to the corresponding move-generation function (pawn_moves, rookMoves, knightMoves, bishopMoves, queenMoves, kingMoves) via a function-dispatch dictionary (self.moveFunctions). This generates pseudo-legal moves without checking for self-check.
2. validMovesFinder() — filters pseudo-legal moves by making each one, testing whether the current player's king ends up in check (via inCheck()), and discarding the move if so.

After filtering, it appends castling moves (which have their own pre-validation) and determines checkmate/stalemate.

The reverse-iteration pattern ('for i in range(len(moves)-1, -1, -1)') is used deliberately when deleting from the list during iteration, since deleting from the back avoids index-shifting bugs — a thoughtful implementation detail.

3.4 Check Detection

inCheck() calls squareUnderAttack(kingRow, kingCol). That function temporarily flips the turn flag, generates all opponent pseudo-legal moves, restores the flag, and checks whether any opponent move's destination matches the king's square. This avoids building a separate attack-map data structure and keeps the code concise.

3.5 Special Moves

All three FIDE special moves are implemented:

3.5.1 En Passant

When a pawn advances two squares, enpassantPossible is set to the skipped square's coordinates. The pawn move generator checks this coordinate when generating diagonal captures. makeMove() handles the physical removal of the captured pawn (which is on the same row as the moving pawn, not the landing square). The enpassantPossibleLog stack ensures this is correctly restored on undo.

3.5.2 Castling

CastleAbility tracks four independent booleans: wks (white kingside), wqs (white queenside), bks (black kingside), bqs (black queenside). change_castle_rights() revokes the appropriate rights whenever a king or rook moves, or whenever a rook is captured. castle_moves() enforces all three FIDE conditions: the king is not currently in check, the path squares are empty, and the transit squares are not under attack. makeMove() physically relocates the rook alongside the king.

3.5.3 Pawn Promotion

Promotion is detected inside the Move constructor by checking whether a pawn reaches the back rank. pawn_moves() uses an inner helper (addPawnMove) that, when isPawnPromotion is true, generates four separate Move objects — one for each legal promotion piece (Q, R, B, N) — so the engine and AI both receive all options as distinct legal moves. The human player is presented with a graphical selection UI in ChessMain.py.

3.6 Move Class

The Move class encapsulates all information about a single move: start and end coordinates, the piece moved, the piece captured, and flags for special move types. A unique integer moveID ($\text{startRow} \times 1000 + \text{startCol} \times 100 + \text{endRow} \times 10 + \text{endCol}$) enables efficient equality comparison via `__eq__()`. The `__str__()` method generates Standard Algebraic Notation (SAN), including castling symbols (O-O, O-O-O), capture notation (x), pawn file disambiguation on capture, and promotion notation (=Q). The `setDisambiguation()` method resolves ambiguous notation when two pieces of the same type can reach the same square, correctly applying file, rank, or full-coordinate disambiguation.

4. AI Engine (chessAI.py)

The AI module contains a progression of algorithms from naive to sophisticated, reflecting the development history of the project. All are still present in the codebase and can be switched by uncommenting the relevant call inside `findBestMove()`.

4.1 Algorithm Progression

Function	Description
<code>randomMoveFinder()</code>	Returns a uniformly random legal move. Used as a fallback if the search process returns None.
<code>findBestMoveMinMaxNoRecursion()</code>	Iterative two-ply minimax (looks one move ahead for each player). No recursion. Evaluates only by material.
<code>findMoveMinMax()</code>	Recursive minimax to arbitrary depth. Uses a global 'nextMove' variable to propagate the root move.
<code>findMoveNegaMax()</code>	Recursive NegaMax reformulation of minimax. Simplifies the code by always maximising from the current player's perspective, negating scores at each recursive step.
<code>findMoveNegaMaxAlphaBeta()</code>	The production algorithm. NegaMax with Alpha-Beta pruning, which prunes branches that cannot improve upon the current best, dramatically reducing the number of nodes evaluated.

4.2 Active Algorithm: NegaMax with Alpha-Beta

The active call in `findBestMove()` is `findMoveNegaMaxAlphaBeta()`. It operates as follows:

3. A depth counter decrements on each recursive call. At depth 0, `scoreBoard()` evaluates the position statically.
4. For each legal move: the move is made, the opponent's best response score is found recursively (negated), and the move is undone.

5. Alpha tracks the best score for the current maximizing player. Beta tracks the opponent's best found so far. When $\alpha \geq \beta$, the remaining sibling branches are pruned since the opponent would never allow the current line.
6. The global variable `nextMove` is updated whenever a new best move is found at the root depth.

The move list is shuffled with `random.shuffle()` before searching. This introduces variation into the AI's play when multiple moves have equal scores, making it less predictable without affecting correctness.

4.3 Evaluation Function: `scoreBoard()`

`scoreBoard()` returns a signed float score, positive favouring white and negative favouring black — matching the convention of chess.com's evaluation bar. It first checks for checkmate (returning ± 1000) and stalemate (returning 0). For all other positions it computes a material-plus-positional score:

For each piece on the board, the base material score is looked up from `pieceScore` and added (white) or subtracted (black). A positional bonus is also added, scaled by 0.1, using piece-square tables (described below).

4.4 Piece-Square Tables

Seven 8x8 integer tables encode positional preferences:

Table	Design Intent
<code>knightLocStrength</code>	Concentric rings from 1 (edge) to 4 (centre); encourages central knight posts.
<code>bishopLocStrength</code>	Inverted pattern rewarding diagonals over central squares, reflecting bishop geometry.
<code>queenLocStrength</code>	Biased toward active central deployment while avoiding premature development.
<code>rookLocStrength</code>	Rewards open files (rank 0 and 7) and penalises passive positions.
<code>whitePawnLocStrength</code>	Directional; encourages pawn advancement with high scores near promotion rank.
<code>blackPawnLocStrength</code>	Mirror of <code>whitePawnScores</code> for black's perspective.
(King table)	No king table; king safety is handled implicitly through checkmate scoring.

4.5 Difficulty and Depth Control

`DEPTH` is a module-level constant defaulting to 3. `chessMenu.py` overrides it before launching the game. The mapping is:

Menu Label	Search Depth	Approximate Character
Easy	1	Looks one move ahead — plays legal moves only.
Medium	2	Two-ply — avoids obvious one-move blunders.
Hard	3	Three-ply — finds most tactical combinations.
Expert	4	Four-ply — strongest but noticeably slower.

5. Rendering & Game Loop (ChessMain.py)

ChessMain.py is the largest module (~950 lines) and is responsible for everything the player sees and hears during a game. It contains the Pygame event loop, all drawing routines, every animation, the promotion selection UI, and the multiprocessing bridge to the AI.

5.1 Layout

The window is 762 px wide and 512 px tall:

- 512 x 512 px — the 8x8 game board (64 px per square)
- 250 x 512 px — the move log panel to the right

The board uses a dark arcane colour scheme: deep purple (28, 18, 45) for dark squares and a softer purple (65, 45, 95) for light squares. A subtle magical glow overlay (fixed star positions + faint vertical beams) is blitted over the board each frame via `drawArcaneOverlay()` without causing per-frame flickering, since the star positions are pre-computed once at module load time.

5.2 Main Game Loop

The `main()` function accepts `playerOne` and `playerTwo` booleans. When both are `True` the game runs in Human vs Human mode with the board automatically flipping each turn so each player views from their own perspective. When one is `False`, that side is controlled by the AI.

Each iteration of the loop:

1. Processes all Pygame events (mouse clicks for move selection, keyboard 'z' for undo, 'r' for reset).
2. If it is the AI's turn and the AI process is not yet running, spawns a new Process targeting `chessAI.findBestMove()`, passing a multiprocessing Queue for the result.
3. If the AI process has finished, collects the result from the Queue, disambiguates it for notation, and calls `makeMove()`.
4. If a move was made, sequences through the animation and sound pipeline (described below).

5. Calls `drawGameState()` to render the full current position.
6. Checks for and displays the end-game overlay on checkmate or stalemate.

Using multiprocessing (as opposed to threading) is deliberate: the GIL in CPython means threads share execution time, but a separate Process gets its own CPU core and avoids blocking the Pygame rendering loop during deep AI searches.

5.3 Input Handling

Mouse input uses a two-click model: the first click selects a piece (highlighted in gold), and the second click selects the destination. Legal destination squares are indicated by semi-transparent dark dots. If the player clicks the same square twice, the selection is cleared. If the second click is not a legal destination, `playerClicks` is reset to `[sqSelected]` (the second click becomes the new first click), allowing natural re-selection.

The board flip in Human vs Human mode is handled by remapping the pixel coordinates: `'row = 7 - row; col = 7 - col'` after detecting that it is black's turn.

5.4 Animation System

The project implements a multi-tier animation system:

5.4.1 Move Animation (`animateMove`)

Interpolates the moving piece between start and destination squares over multiple frames using a smoothstep easing function ($t * t * (3 - 2t)$) for natural deceleration. A 'fast' mode (used when the move gives check) uses linear interpolation and renders ghost trail frames behind the piece to convey speed.

5.4.2 Capture Effects

Each piece type triggers a distinct visual effect on capture, dispatched through a dictionary keyed by piece type:

Piece	Effect
Pawn (p)	Expanding shockwave ring + central flash + particle burst (warm gold)
Knight (N)	Shrinking/rotating captured piece sprite + emerald ring + particles
Bishop (B)	X-shaped diagonal beams + central flash + particles (electric violet)
Rook (R)	Cross-shaped straight-line blast + double concentric rings + particles (steel blue)
Queen (Q)	Eight radial beams in all queen directions + supernova flash + halo + dense particles (royal gold)
King (K)	Expanding ring + particles (crimson) — triggered only in informal play

5.4.3 Check Effects

`animateCheckEffect()` highlights the king's square with a pulsing red overlay and draws red beam lines from each checking piece to the king, animating for 54 frames.

`animateResolveCheckEffect()` plays a blue expanding ring when a player escapes check without capturing.

5.4.4 Promotion Effect

`animatePromotionEffect()` plays a golden sparkle burst with gravity-affected particles and periodic bright flashes over 45 frames.

5.5 Sound System

Five sound events are loaded at startup via `loadSounds()`, with graceful `FileNotFoundError` handling so the game runs even if audio assets are missing:

Event	Trigger Condition
move	Any non-capture, non-promotion move
capture	Any capturing move
check	A move that leaves the opponent in check (but not checkmate)
checkmate	The game-ending checkmate move
promotion	A pawn reaching the back rank

5.6 Move Log Panel

The right-hand panel renders the game's move history in Standard Algebraic Notation formatted as numbered pairs (e.g. '1. e4 e5 2. Nf3 ...'). Alternating rows receive a subtle purple tint. The panel uses the Georgia typeface and is headed 'WIZARD'S LOG' to match the game's theme.

5.7 Promotion Selection UI

`drawPromotionUI()` overlays a darkened panel in the centre of the board presenting the four promotion choices (Q, R, B, N) as clickable piece images. The function blocks in its own event loop until the player clicks a valid piece or closes the window (defaulting to Queen). The selected piece type is then applied to the move before it is executed.

6. Menu System (chessMenu.py)

`chessMenu.py` provides the application's entry point and serves two screens: the home screen and the AI settings screen.

6.1 Home Screen

Displays the title 'WIZARD'S CHESS' with a shadow offset for depth, and three buttons: 'Play vs Human', 'Play vs AI', and 'Quit'. Button hover states are detected via `Rect.collidepoint()` and render with a brighter background and a purple glow halo. The same arcane overlay (stars + beams) used on the game board is applied here for visual consistency.

6.2 AI Settings Screen

Allows the player to choose their colour (White, Black, or Random) and a difficulty level (Easy, Medium, Hard, Expert). Selected options are highlighted in green. A description of the selected difficulty and its search depth is displayed below the buttons. 'Random' colour is resolved to White or Black using `random.choice()` immediately before launching the game. The difficulty selection sets `chessAI.DEPTH` directly before `ChessMain.main()` is called.

6.3 Music Management

Background music (`Chess/sounds/titular.mp3`) is loaded and looped at 40% volume during the menu. `stopBackgroundMusic()` is called before transitioning to the game so the game's own sound effects play without competition. Both functions handle exceptions silently so a missing audio file does not crash the menu.

7. Design Strengths

- Complete rules implementation: all FIDE rules, including en passant, both forms of castling, and pawn promotion with all four choices, are correctly implemented.
- Clean separation of concerns: game logic, AI, rendering, and navigation are strictly partitioned into separate modules with one-directional imports.
- Multiprocessing AI: the AI runs in a separate process rather than blocking the UI thread, keeping the game responsive during search.
- Algorithm progression documented in code: MinMax, NegaMax, and Alpha-Beta are all present, making the codebase a readable study of chess AI development.
- Contextual visual feedback: distinct per-piece capture animations, check beam effects, and promotion sparkles give the game a high degree of visual polish relative to its scope.
- Board flip for Human vs Human: the board automatically reorients for Black's perspective, which is a common omission in amateur chess implementations.
- Graceful degradation: missing audio files are caught and skipped; missing AI results fall back to a random move; the promotion UI defaults to Queen on window close.
- Standard Algebraic Notation with disambiguation: the move notation implementation is correct and handles the rare three-piece disambiguation case.

8. Known Limitations & Possible Improvements

8.1 Current Limitations

- No draw-by-repetition or fifty-move-rule detection: the game can loop indefinitely in positions where neither side can force a win.
- Flat move log: the log panel does not scroll, so games longer than roughly 25 moves overflow the visible area.
- No time control: there is no clock, making the game unsuitable for timed play.
- No persistent game saving: a game in progress cannot be saved and resumed.

8.2 Suggested Improvements

- Move ordering: sorting the move list by captures, checks, and killer moves before the Alpha-Beta loop would reduce the effective branching factor and allow deeper search at the same difficulty levels.
- Scrollable move log: replacing the static text panel with a scrolling widget would support full-game notation display.
- Draw detection: implementing repetition detection (via a position hash log) and the fifty-move rule would make the game rule-complete.
- Opening book: pre-loading a small database of standard openings would improve the AI's early-game play quality without additional search cost.
- Board themes and piece sets: the visual system's colour constants are already well-structured, making theme support a relatively minor extension.

9. Conclusion

Wizard's Chess is a well-structured, feature-rich chess implementation that goes substantially beyond a basic board display. The codebase demonstrates a solid grasp of object-oriented design, recursive algorithm implementation, Pygame rendering techniques, and multiprocessing in Python. The game is rule-complete for standard play, supports two game modes, provides a configurable AI opponent powered by Alpha-Beta pruning, and wraps everything in a cohesive visual and audio theme. The main avenues for improvement — draw detection, move ordering, and a transposition table — are standard extensions in chess engine development and would be natural next steps as the project matures.

10. References

1. <https://docs.python.org/3/library/index.html>
2. <https://www.fide.com/FIDE/handbook/LawsOfChess.pdf>
3. CPE 204 Lectures by Dr. Emmanuel Ali
4. <https://www.geeksforgeeks.org/artificial-intelligence/mini-max-algorithm-in-artificial-intelligence/>
5. <https://www.geeksforgeeks.org/artificial-intelligence/alpha-beta-pruning-in-adversarial-search-algorithms/>

Thank You 🙏